

01

Install Node.js and MongoDB

You need both Node.js for the server and MongoDB for the database.

- Download and install **Node.js** from the official site
- Install **MongoDB Community Edition** or use **MongoDB Atlas** (cloud) if you prefer

02

Create Project Folder

Organize your files for the application.

- Make a folder named `employee-crud`
- Inside, add `app.js` and a `routes` folder with `employees.js`

03

Initialize npm

Set up Node.js package management.

Run in terminal: `npm init -y`

- This creates a `package.json` file
- It tracks dependencies and scripts

04

Install Dependencies

Install required libraries.

Run in terminal: `npm install express mongoose`

- **express** → web server framework
- **mongoose** → MongoDB driver

05

Start MongoDB

Ensure your database is running.

- If local: run `mongod service`
- If cloud: create a cluster in MongoDB Atlas and copy the connection string

06

Update Connection String

Connect your app to MongoDB.

- In `app.js`, replace `'mongodb://localhost:27017/employeeDB'` with your Atlas URI if using cloud
- Keep `employeeDB` as the database name

07

Run the Server

Start your Node.js application.

Run in terminal: `node app.js`

- You should see ☒ Connected to MongoDB
- And 🚀 Server running at `http://localhost:3000`

08

Test Endpoints

Verify CRUD operations with Postman or curl.

- `POST /employees` → add employee
- `GET /employees/:employeeID` → retrieve employee
- `PUT /employees/:employeeID` → update employee
- `DELETE /employees/:employeeID` → remove employee

Sources: Express.js and Mongoose official documentation

app.js

```
const express = require('express');
```

```
const mongoose = require('mongoose');
```

```
const employeeRoutes = require('./routes/employees');
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// Middleware
```

```
app.use(express.json());

// MongoDB connection
mongoose.connect('mongodb://localhost:27017/employeeDB', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('✅ Connected to MongoDB'))
.catch(err => console.error('❌ MongoDB connection error:', err));

// Routes
app.use('/employees', employeeRoutes);

// Start server
app.listen(PORT, () => {
  console.log(`🚀 Server running at http://localhost:${PORT}`);
});
```

routes/employees.js

```
const express = require('express');
const router = express.Router();
const mongoose = require('mongoose');

// Employee Schema
const employeeSchema = new mongoose.Schema({
  employeeID: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  designation: { type: String, required: true }
});

const Employee = mongoose.model('Employee', employeeSchema);
```

//  Add new employee

```
router.post('/', async (req, res) => {  
  try {  
    const employee = new Employee(req.body);  
    await employee.save();  
    res.status(201).json({ message: 'Employee added successfully', employee });  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

//  Retrieve employee(s) by employeeID

```
router.get('/:employeeID', async (req, res) => {  
  try {  
    const employee = await Employee.findOne({ employeeID: req.params.employeeID });  
    if (!employee) return res.status(404).json({ message: 'Employee not found' });  
    res.json(employee);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

//  Update employee details

```
router.put('/:employeeID', async (req, res) => {  
  try {  
    const employee = await Employee.findOneAndUpdate(  
      { employeeID: req.params.employeeID },  
      req.body,  
      { new: true }  
    );  
  }  
});
```

```
    if (!employee) return res.status(404).json({ message: 'Employee not found' });  
    res.json({ message: 'Employee updated successfully', employee });  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

// 🗑 Delete employee

```
router.delete('/:employeeID', async (req, res) => {  
  try {  
    const employee = await Employee.findOneAndDelete({ employeeID: req.params.employeeID });  
    if (!employee) return res.status(404).json({ message: 'Employee not found' });  
    res.json({ message: 'Employee deleted successfully' });  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});  
  
module.exports = router;
```